

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**IV SEMESTER B.TECH. CSE****MID SEMESTER EXAMINATION MARCH 2025****SUBJECT: EMBEDDED SYSTEMS [CSE 2223]**

Time: 8.30 AM – 10.00 AM

Date: 03/03/2025

MAX.MARKS: 30

Note: Missing data may be assumed suitably. Duration: 15 minutes

Q. No.	Questions	Marks	CLO	AHEP LO	BL
1.	Identify the amount of memory occupied within the address range 0x60000000 – 0x6001FFFF for SRAM 1. 0x1FFFF KB 2. 128 MB 3. 128 B 4. **128 KB	0.5	1	1	3
2.	The ability to shift or rotate in the same instruction along with other operations is performed with the help of _____ 1. Switching circuit 2. **Barrel shifter 3. Integrated Switching circuit 4. Multiplexer circuit	0.5	2	1	2
3.	Identify the operation that does not exist for the ADD instruction in RISC. 1. Register to Register 2. **Memory to Memory 3. Immediate to Register 4. None of the given options	0.5	1	1	3
4.	The BEQ instruction is used _____ 1. To check the inequality condition between operands and then branch 2. To check if the operand is greater than the condition value and then branch 3. **To check if the flag Z is set to 1 and then causes branch 4. None of the other options	0.5	2	1	2
5.	The register which holds the address of the instruction to be executed next is _____ 1. **R15 2. R14 3. R13 4. R12	0.5	1	1	1
6.	Identify which of the following address/es is/are word aligned	0.5	1	1	3

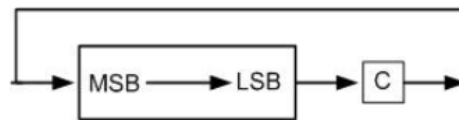
	i) 0x52000068 ii) 0x66000082 iii) 0x18000014 iv) 0x1200004A 1. ** i) and iii) only 2. ii) and iv) only 3. ii) and iii) only 4. (i), (ii), (iii), and (iv)				
7.	Identify the ARM instruction that will conditionally move data from R1 to R2 only if the Zero (Z) flag is set. 1. MOV R2, R1 2. MOVNE R2, R1 3. ** MOVEQ R2, R1 4. CMN R1, R1	0.5	2	1	3
8.	What is the result of the following ARM instruction? EOR R1, R2, #0xFF 1. R1 = R2 AND 0xFF 2. R1 = R2 OR 0xFF 3. R1 = R2 NOR 0xFF 4. ** R1 = ~R2	0.5	2	1	3
9.	Identify the statement(s) that can be used to clear the values in port pins 1.11 to 1.16? i) LPC_GPIO0->FIOSET=0x1F800 ii) LPC_GPIO0->FIOCLR=0x1F800 iii) LPC_GPIO1->FIOPIN=0x0 iv) LPC_GPIO1->FIOCLR=0x1F800 1. i and ii only 2. ii and iii only 3. ** iii and iv only 4. i and iv only	0.5	4	6	3
10.	The purpose of the FIOxMASK register is _____. 1. To store the pin's logic state 2. ** To enable or disable access to specific GPIO pins 3. To configure alternate functions 4. To generate an interrupt	0.5	4	6	2
11	Develop an ARM Assembly Language program that performs the following operations: (i) Read a 16-bit value from memory location 0x20001000. (ii) Swap the upper and lower bytes of this value. (iii) Store the modified value back at 0x20001004. Provide comments to explain your logic.	4	2	1	6
AREA SWAP_BYTES, CODE, READONLY ENTRY LDR R0, =0x20001000 ; Load address of 16-bit value					

LDRH R1, [R0] ; Load 16-bit value into R1 MOV R2, R1, LSL #8 ; Shift lower byte to upper position MOV R3, R1, LSR #8 ; Shift upper byte to lower position ORR R2, R2, R3 ; Merge the swapped bytes LDR R4, =0x20001004 ; Load destination address STRH R2, [R4] ; Store swapped value END					
12	Develop an Embedded C Program to implement 4-bit binary up counter in the most significant 4 LEDs of the LED block in LPC1768 when the switch SW2 is pressed. Assume that the LEDs are connected to the port pins from P0.15 to P0.22 and the switch SW2 is connected to P2.0.	4	4	6	6
<pre> #include <LPC17xx.h> int main(void) { unsigned int j; unsigned long LED = 0x00000000; //Configure Port0 PINS P0.15-P0.22 as GPIO function LPC_PINCON->PINSEL0 &= 0x3FFFFFFF; LPC_PINCON->PINSEL1 &= 0xFFFFC000; LPC_PINCON->PINSEL4 &= 0xFFFFFFF0; //Configure P0.15-P0.22 as output port LPC_GPIO0->FIODIR = 0x007F8000; //(or) 0x780000 (or) 0xF<<19 LPC_GPIO2->FIODIR = 0x00000000; while(1) { k = LPC_GPIO2->FIOPIN;// Read the input from 2.12 k &= 0x00000001; if(k==0) { for(LED=0;LED<16;LED++) //On the LED's serially { LPC_GPIO0->FIOPIN = LED<<19;// Turn ON LED at MSB(LEDs connected from p0.19 to 0.22) for(j=0;j<100;j++);//a random delay } // loop for 16 times } } } </pre> Structure & Delay: 1M					

13	Develop an Embedded C Program to display the text “CSE” in the seven-segment display (SSD) unit continuously. The text should be displayed in units U11, U10 and U9 of the SSD unit. Assume that the eight segments of all the SSD units are connected to the port pins P0.4 to P0.11 and the SSD units can be enabled one by one using the port pins P0.15 to P0.18.	4	4	6	6
<pre> #include <LPC17xx.h> unsigned int i=0x00; unsigned char array_dec[3]={0x39,0x6D,0x79}; void delay(void); int main(void) { LPC_GPIO0->FIODIR = 0xFF<<4; //P0.4 to P0.11 output LPC_GPIO0->FIODIR = 0xF<<15; //P0.15 to P0.18 output while(1) { delay(); for(i=1;i<4;i++) { LPC_GPIO0->FIOPIN = i<<15; LPC_GPIO0->FIOPIN = array_dec[i-1]<<4; } } //end of while(1) } //end of main void delay(void) { unsigned int i; for(i=0;i<10;i++); } </pre> Structure & Delay: 1M					
14	Identify the memory addresses referenced by each of the instructions in the following code snippet. The instructions are executed sequentially and the initial values in the registers are as follows: R2 = 0X2000, R3 = 0XFF, and R4 = 0X1000. <pre> LDR R1, [R2, R3, LSL #2] STR R1, [R4, R3] LDRH R1, [R2, #4] STRH R1, [R4, #2]! LDRB R1, [R4], #1 STRB R1, [R2] </pre>	3	1	1	3

R3 LSL 2 = 0x3FC 1) R2+R3 = 0x2000+0x3FC = 0x23FC 2) R4+R3 = 0x10FF 3) R2 + 4 = 0x2004 4) R4=R4+2 = 0x1002 5) 0x1002 6) 0x2000					
15	Discuss the functionalities of FIOMASK register with an example.	3	4	6	6
- This register is used to select port pins that will and will not be affected by write accesses to the FIOxPIN, FIOxSET or FIOxCLR register. Mask register also filters out port's content when the FIOxPIN register is read. 1M - A zero in this register's bit enables access to the corresponding physical pin via a read or write access. If a bit in this register is one, corresponding pin will not be changed with write access and if read, will not be reflected in the updated FIOxPIN register. 1M Example: 1M FIOSET = 0 FIOMASK= 4 FIOSET=0xF The bit 2 in FIOSET remain unchanged due to masking					
16	Assume the values in Registers R5 – R8 are 0x205 to 0x208 respectively. The value of the Stack Pointer (R13) is 0x40002000. Use appropriate instructions to perform push and pop operations on fully ascending and empty descending stacks. Explain the operation with neat diagrams. I. Store the values from R5 – R8 onto the stack. II. Restore the values into R5 – R8 after the push operation. Note: (Use LDM and STM with appropriate suffixes other than EA, FA, ED, and FD suffixes).	3	3	1	4
<u>Full Ascending Stack (FA Not Allowed → Use IB Instead)</u> STMIB SP!, {R5-R8} ; Push R5-R8 using Increment Before (IB) LDMDA SP!, {R5-R8} ; Pop R5-R8 using Decrement After (DA) -1M- <u>Empty Descending Stack (ED Not Allowed → Use DB Instead)</u> STM DA SP!, {R5-R8} ; Push R5-R8 using Decrement After (DA) LDMIB SP!, {R5-R8} ; Pop R5-R8 using Increment Before (DB) -1M- Diagram -1M-					
17	Explain the syntax and working of the instruction RRX in ARM assembly language. Show the steps and the result while performing RRX on the data 0x85.	2	2	1	3

RRX rotate right through carry



In RRX, as bits are shifted from left to right, they exit from the right end (LSB) to the carry flag, and the carry flag enters the left end (MSB). In other words, in RRX the LSB is moved to C and C is moved to the MSB. In reality, C flag acts as if it is part of the operand. That means the RRX is like 33-bit register since the C flag is 33th bit. The RRX takes no arguments and the number of times an operand to be rotated is fixed at one. **-1M-**

Step-by-Step Execution of RRX 0x85, 4 Times

Step	Before (Binary)	Carry (C)	After RRX (Binary)	Result (Hex)
Initial	1000 0101	1 (given)	-	0x85
1st RRX	1000 0101	1	1100 0010	0xC2
2nd RRX	1100 0010	0	0110 0001	0x61
3rd RRX	0110 0001	1	1011 0000	0xB0
4th RRX	1011 0000	0	0101 1000	0x58

-1M-

18	Compare the AND and BIC instructions with examples.	2	2	1	4
BIC(bit clear) - BIC Rd, Rn, Op2 ; The BIC instruction performs AND operation on Rn register with the complement of Op2 and places the result in destination register - clear certain bits of Rn specified by Op2 & place the result in Rd. - The bits that are HIGH in Op2 will be cleared and bits with LOW will be left unchanged.		AND - AND Rd, Rn, Op2 ;Rd = Rn ANDed Op2 - Can be used for masking - This instruction will perform a bitwise logical AND on the operands and place the result in the destination.			

Abbreviations:

M – Marks

CO/CLO– Course Outcome (NBA)/Course Learning Outcome (IET).

CLO – Course Learning Outcome as per AHEP 4

BL - Blooms Taxonomy Level